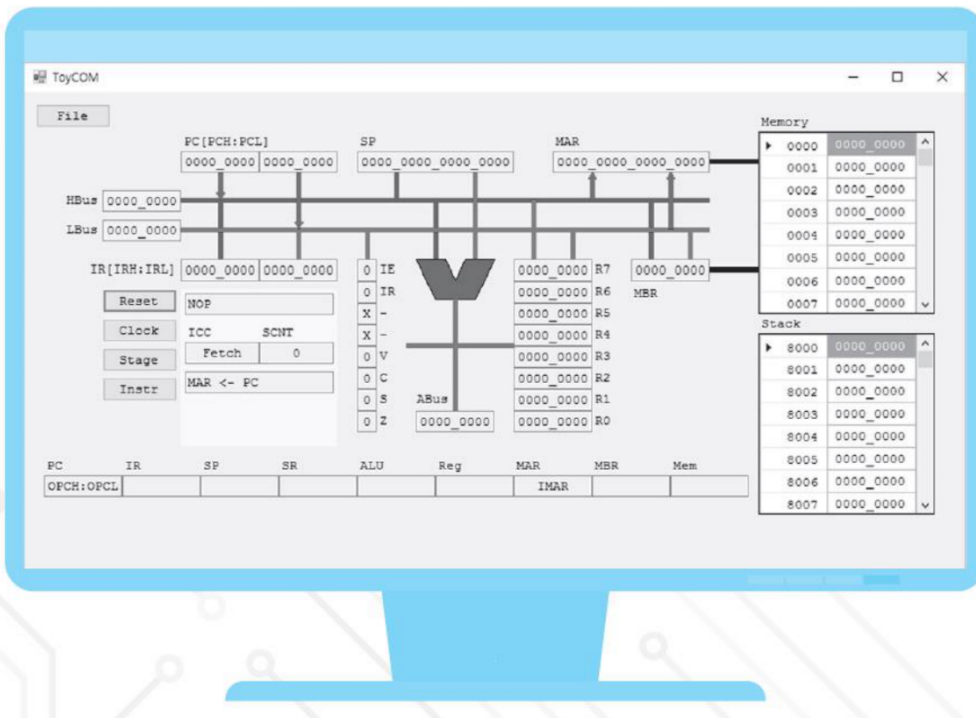


한눈에 보이는

개정 3 판

컴퓨터 구조



03. 컴퓨터 구성 요소

3.1 프로그램 실행

3.2 컴퓨터 구성 요소

3.3 시스템 버스

3.4 명령어

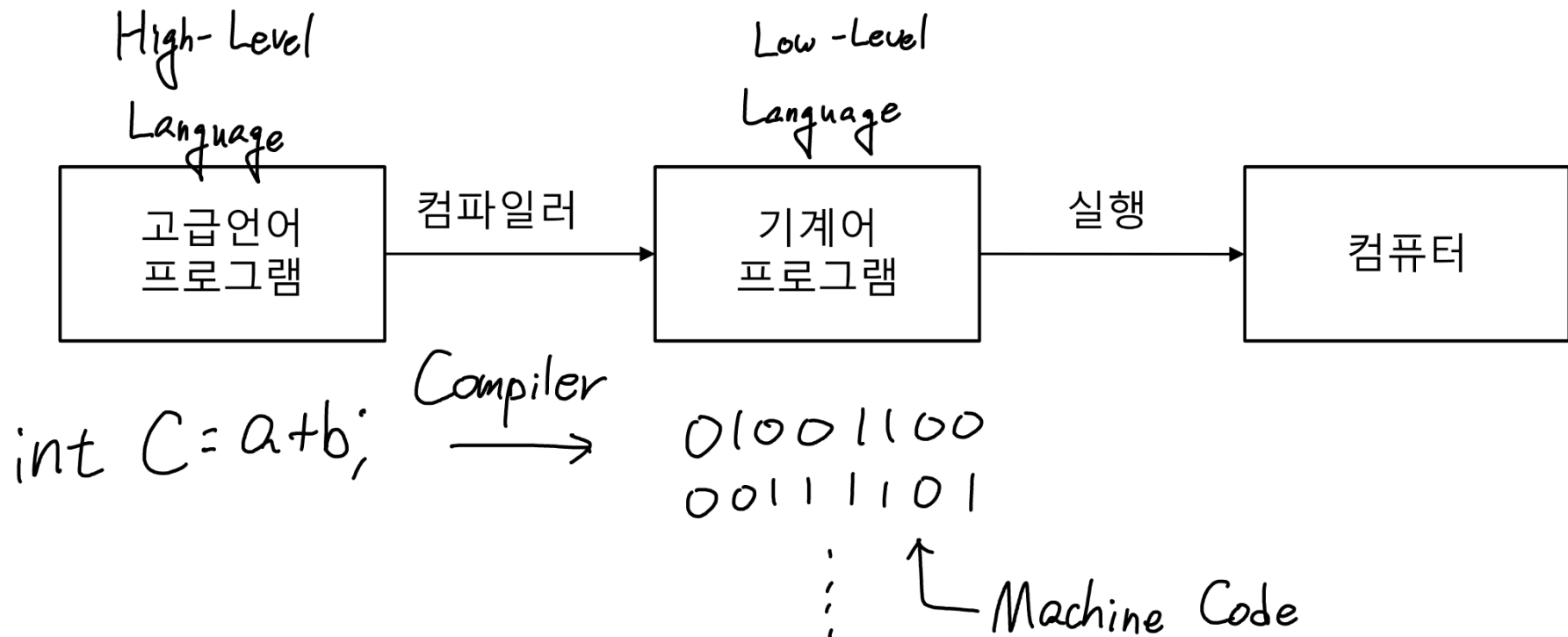
3.5 명령어 실행 과정

3.6 요약

3.1.1 기계어 프로그램

컴퓨터

- 프로그램을 실행하는 기계
- 프로그램은 기계어(machine instructions)의 모임
- 기계어 명령어를 실행하는 기계



고급언어 프로그램과 기계어 프로그램

기계어 (machine language)

- 명령어(instruction)는 이진수 코드
- 명령어 종류별로 길이가 다를 수 있다.

eax
+) ebx: intel 만의 고유한 레지스터 이름
ecx
edx

어셈블리 언어와 어셈블러

- 의미를 쉽게 알 수 있도록 어셈블리 언어로 표현한다.
- 어셈블러는 어셈블리 언어 명령어를 기계어로 번역한다.

	address	machine code	assembly language
1. int n, sum;	0041138E	c745f8000000001	mov dword ptr [n], 1 = n에 1을 넣어라
2. n = 1;	00411395	c745ec000000000	mov dword ptr [sum], 0
3. sum = 0;	0041139C	837df864	cmp dword ptr [n], 64h (=100) = n과 100을 비교하라
4. while (n <= 100) {	004113A0	7f14	jg wmain+46h (4113B6h) = n > 64h 이면 while 밖으로 점프하라
5. sum = sum + n;	004113A2	8b45ec	mov eax, dword ptr [sum]
6. n = n + 1;	004113A5	0345f8	add eax, dword ptr [n]
7. }	004113A8	8945ec	mov dword ptr [sum], eax
	004113AB	8b45f8	mov eax, dword ptr [n]
	004113AE	83c001	add eax, 1
	004113B1	8945f8	mov dword ptr [n], eax
	004113B4	ebe6	jmp wmain+2Ch (41139Ch) = 해당 주소로 점프하라

Pentium

AVR 기계어 프로그램

	address	machine code	assembly language
	1. 000067	e001	LDI R16, LOW(1)
	2. 000068	e010	LDI R17, HIGH(1)
	3. 000069	e020	LDI R18, LOW(0)
1. int n = 1;	4. 00006a	e030	LDI R19, HIGH(0)
2. int sum = 0;	5. 00006b	3605	_0x3: CPI R16, LOW(101)
	6. 00006c	e0e0	LDI R30, HIGH(101)
3. while (n <= 100) {	7. 00006d	071e	CPC R17, R30
4. sum = sum + n;	8. 00006e	f42c	BRGE _0x5
5. n = n + 1;	9. 00006f	0f20	ADD R18, R16
6. }	10. 000070	1f31	ADC R19, R17
	11. 000071	5f0f	SUBI R16, LOW (-1)
	12. 000072	4f1f	SBCI R17, HIGH (-1)
	13. 000073	cff7	RJMP _0x3
	14. 000074		_0x5:

Atmega 128

3.1.2 프로그램 내장형 컴퓨터

프로그램 내장형(stored program) 컴퓨터

- Von Neumann 형 컴퓨터
- 프로그램과 데이터를 주기억장치에 저장한다.
- 명령어를 한 개씩 중앙처리장치로 가져와 실행한다.

기억장치 구조

- 일반 구조: 한 개의 기억장치에 프로그램과 데이터를 함께 저장한다.
- 하버드 구조: 프로그램과 데이터를 별도의 기억장치에 저장한다.

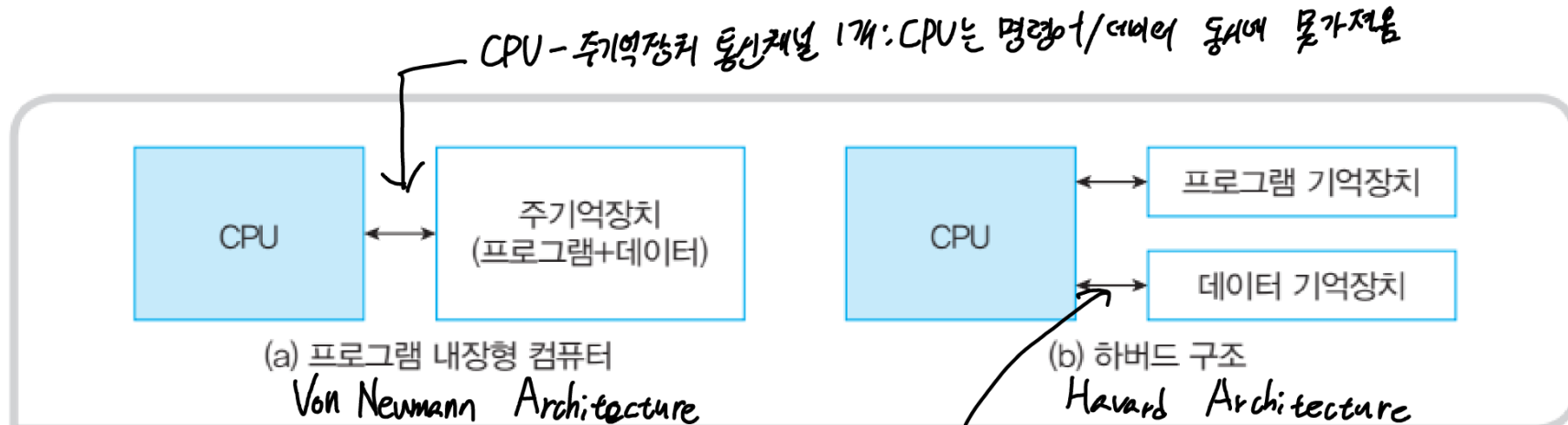


그림 3-2 프로그램 내장형 컴퓨터의 개념

프로그램/데이터 분리 : 동시에 액세스 가능
↳ 속도 ↑

명령어 사이클

fetch : CPU가 메모리에서 읽는 과정

명령어 사이클 = {인출단계, 실행단계}
instruction cycle *fetch stage* *execute stage*
인출단계(fetch stage)

CPU: 전원 인가 시 인출 → 실행 → 인출
무한반복

- 기억장치에서 명령어를 중앙처리장치로 가져온다.

실행단계(execution stage)

- 중앙처리장치에서 명령어를 해독하여 실행한다.

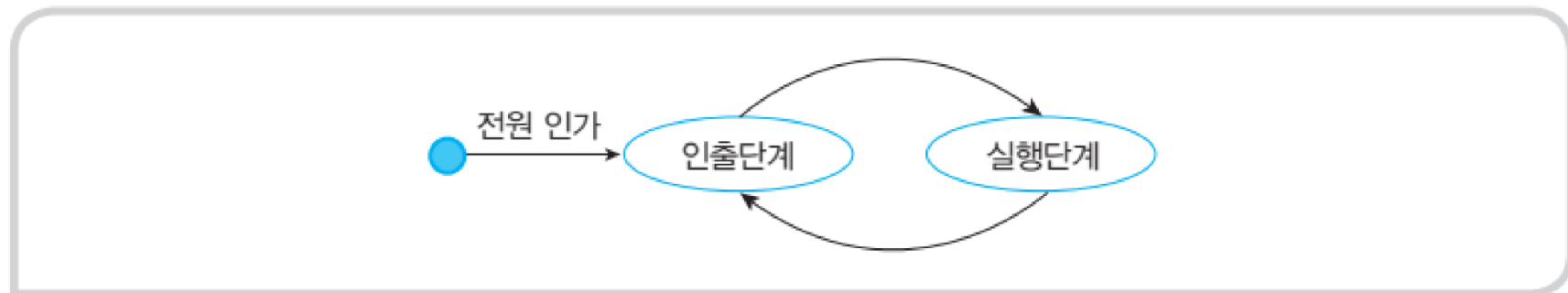


그림 3-3 명령어 사이클

CPU : 프로그램 실행, 컴퓨터 모든 동작 제어

주기억장치 : 프로그램/데이터 저장

입출력장치 : 컴퓨터 내외부 데이터 전송

시스템버스 : 요소들 간 고속도로 역할

3.2 컴퓨터 구성 요소

학습 목표

- 컴퓨터의 세 가지 구성 요소인 중앙처리장치, 주기억장치, 입출력 장치의 기능을 이해한다.

CPU : 프로그램 실행, 컴퓨터 모든 동작 제어

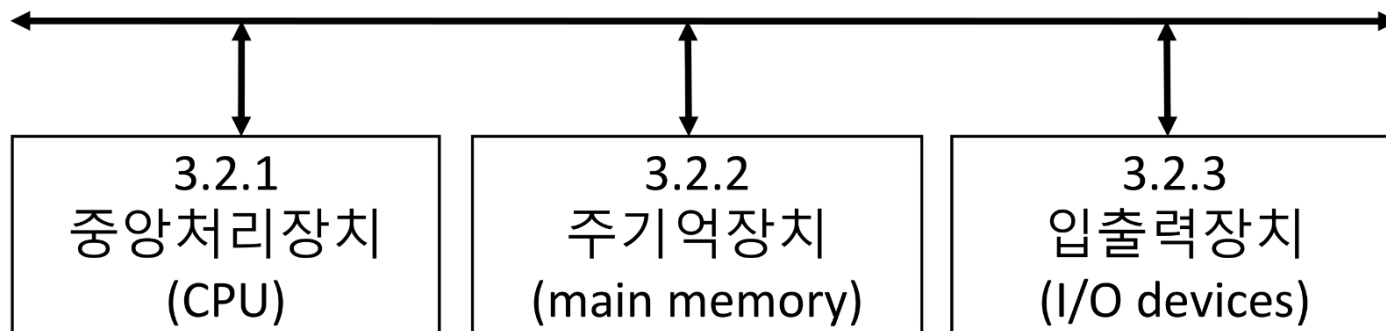
주기억장치 : 프로그램/데이터 저장

입출력장치 : 컴퓨터 내외부 데이터 전송

시스템버스 : 요소들 간 고속도로 역할

내용

3.3 시스템 버스



포인터 → PC : 다음 명령어의 기억 주소 저장

명령어 인출 후 그 다음 명령어 주소를 가리키도록 설정

3.2.1 중앙처리장치

IR: fetch 에서 PC가 지정한 기억주소에서 fetch한 명령어 저장
현재 명령어 저장

인출 단계

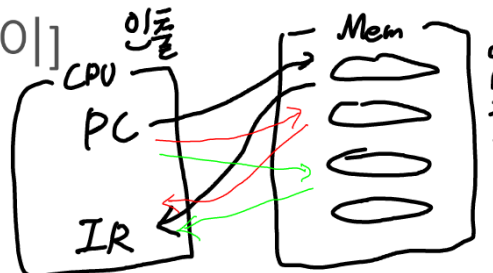
기억장치에서 명령어를 하나씩 가져온다.

☆ 구성 요소 ☆

- 특수용 레지스터
 - PC (Program Counter) CPU 내부에 존재
 - IR (Instruction Register) Flip-Flop으로 구성

동작

- IR $\leftarrow$ Mem[PC]
- PC $\leftarrow$ PC + [명령어의 길이]



실행 단계

IR의 명령어를 해석하여 실행한다.

구성 요소

- 제어장치(Control Unit) 명령어 해독 CU
- 내부 레지스터 (internal registers)
- 연산기(Arithmetic and Logic Unit)
- 명령어 처리: Processing Unit ; PU

동작

- 명령어마다 하는 일이 다르다.

PC → Mem → IR 반복

중앙처리장치 구조

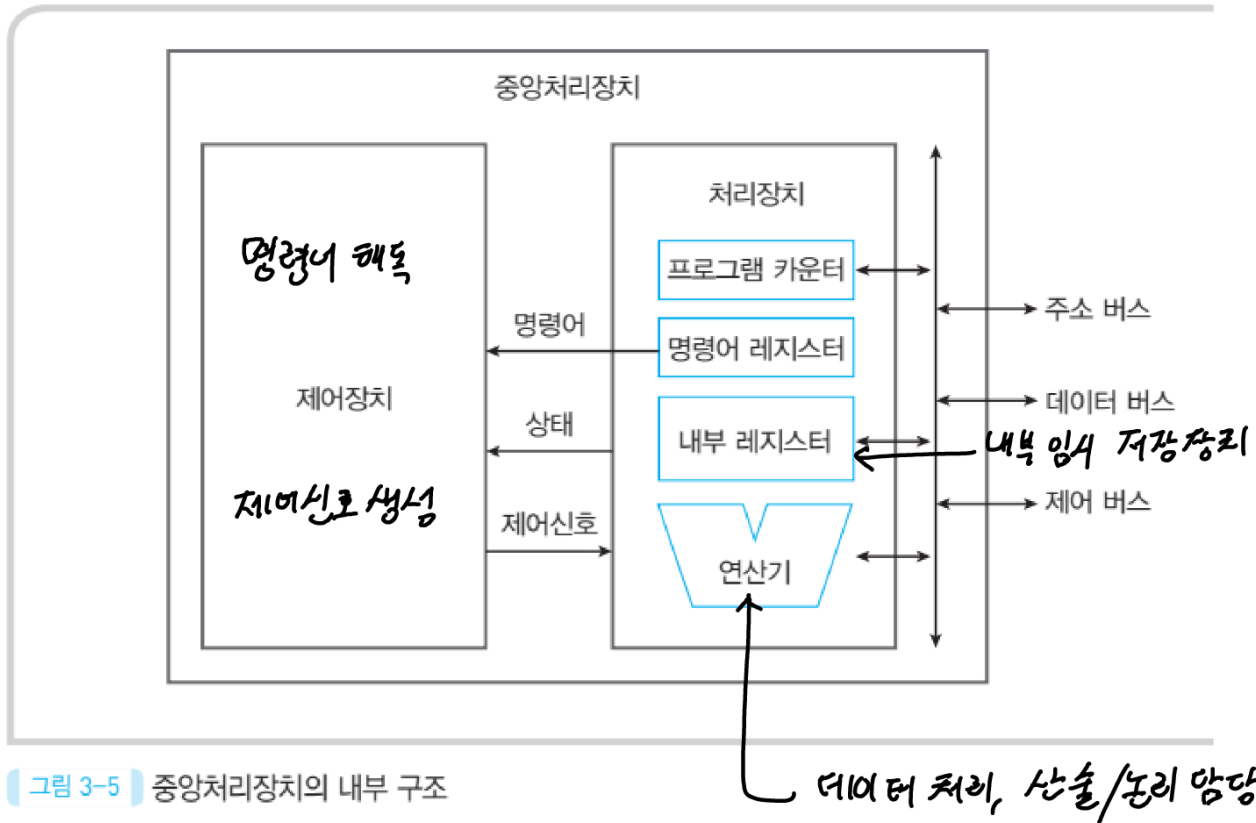
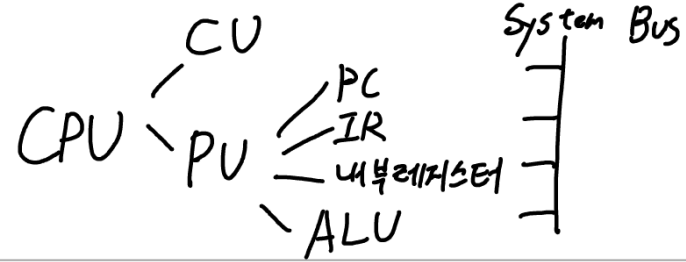


그림 3-5 중앙처리장치의 내부 구조

PC

- 다음에 실행할 명령어 주소 저장

IR

- 현재 실행 중인 명령어 저장

제어장치

- 중앙처리장치 내부와 외부 제어신호 생성

내부 레지스터

- 데이터 임시 저장

연산기

- 산술연산, 논리연산

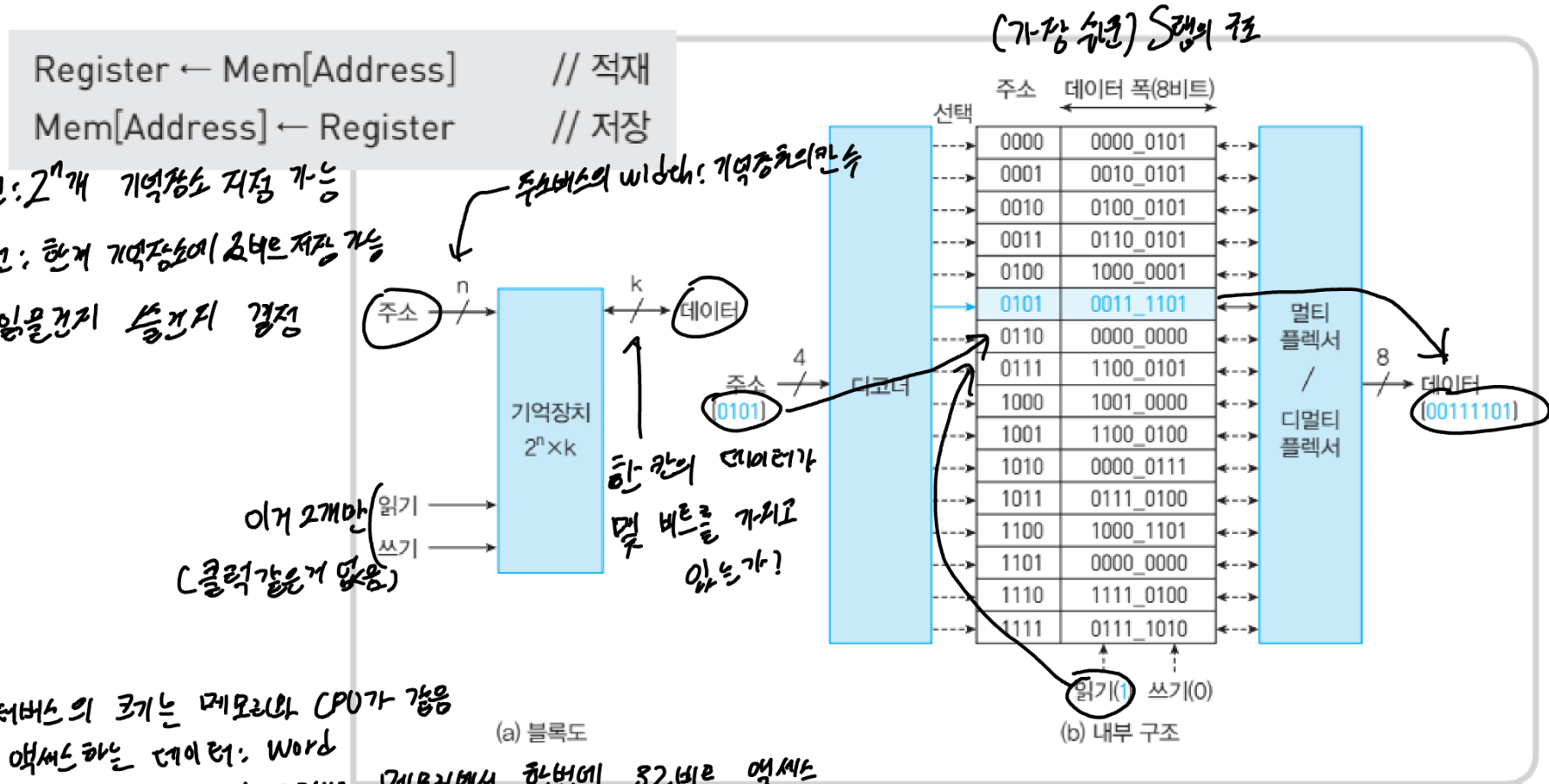
<CPU 관점>

Access 액세스: load + store
 load: Read. 데이터 가져오기
 store: Write. 데이터 입력하기

3.2.2 주기억장치

프로그램과 데이터를 저장하는 거대한 1차원 배열

기억장치 용량 = $2^n \times k$ 비트 (n 비트 주소선, k 비트 데이터선)

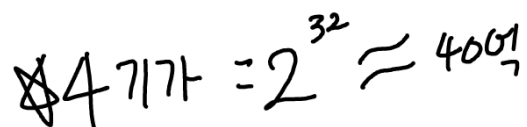


일반적으로 데이터버스의 크기는 메모리와 CPU가 같음
 CPU가 한 번에 액세스하는 데이터: word
 32bit CPU: 레지스터 32비트, 연산기 32비트, 메모리에서 한 번에 32비트 액세스
 Word: CPU마다 다름

그림 3-6 기억장치의 구조

[예제 3-1] 기억장치 동작

- 0101 0011-1101 2421
1101-0000 2421



int : 4바이트 = 32비트 $\Rightarrow$ $2^{31} \sim 2^{32}$

$$16\text{MiB} : 16 \text{ 메가바이트}$$
$$= 16 \times 2^{20} \text{ 바이트}$$

- [예제 3-2] 기억장치 주소선 수와 데이터선 수는?

- 1) 16M×16비트 주소선 = 24 비트, 데이터선 = 16 비트
- 2) 4G×8비트 주소선 = 32 비트, 데이터선 = 8 비트

3.2.3 입출력장치

기능

- 프로그램과 데이터를 공급한다.
- 컴퓨터가 처리한 데이터를 출력한다.

특징

- 종류가 다양하고, 기계 동작을 포함한다.
- 중앙처리장치와 주기억장치에 비하여 동작 속도가 느리다.

예

- 입력장치: 키보드, 마우스, 마이크, 카메라, 터치 스크린, 스캐너
- 출력장치: 모니터, 프린터, 플로터, 스피커
- 입출력장치: 보조 기억장치(디스크, USB 메모리 등), 네트워크 어댑터

컴퓨터는 입출력장치를 구별하기 위해 장치마다 주소 할당

입출력장치 구조

→ I/O Address or I/O port
입출력 주소 입출력 포트

CPU 입장에서 메모리와 입출력장치는 구동 원리가 같음

인터페이스 (interface)

- 데이터 전송 속도, 데이터 형식 조정

내부 구조

신호변환기 (transducer)

- 전기 신호를 기계 동작으로 변환 또는 반대

입력 : 키/마, 마이크 등
↓
출력 : 모니터, 플로터 등

기계장치, 동작부 (actuator)

- 기계 구동

입출력 : USB 등

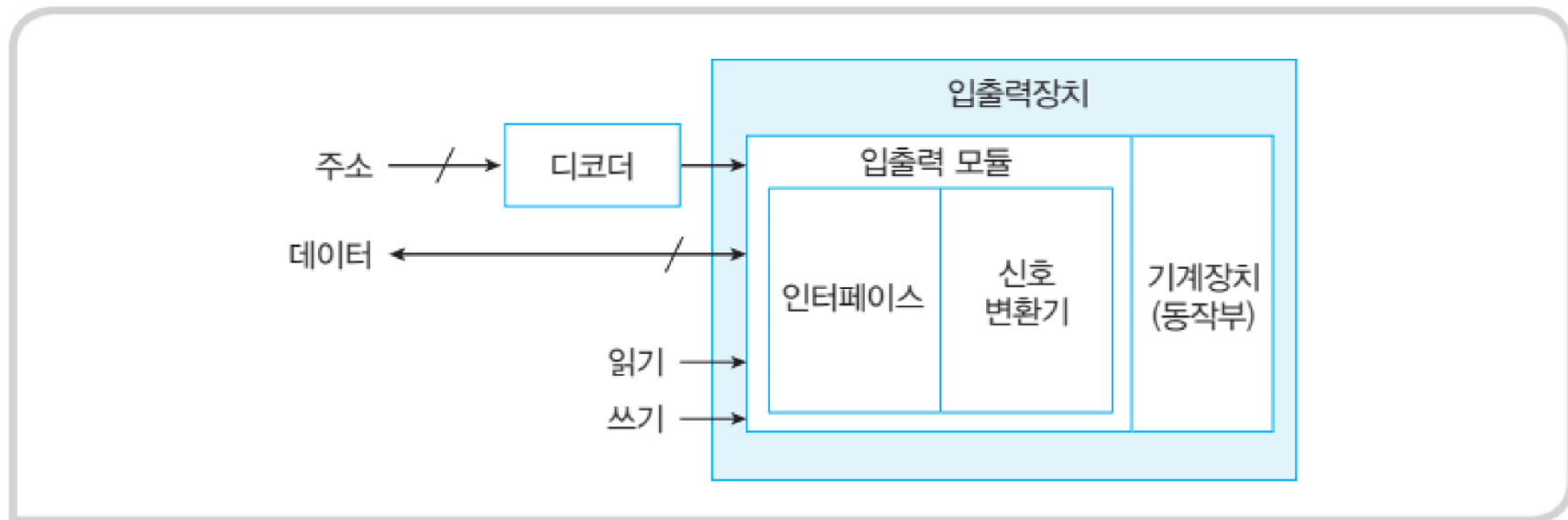


그림 3-7 입출력장치 구조

3.2.4 AVR 마이크로제어기 = MCU (Micro Controller Unit)

마이크로제어기 $\rightarrow$ 마이크로컨트롤러 $\rightarrow$ = CPU + 메모리 + ...

- 하나의 칩에 중앙처리장치, 기억장치, 주변장치를 포함한 반도체 소자
- 소형 임베디드 시스템의 제어장치로 쓰임.

AVR 마이크로제어기: 아두이노 제어기

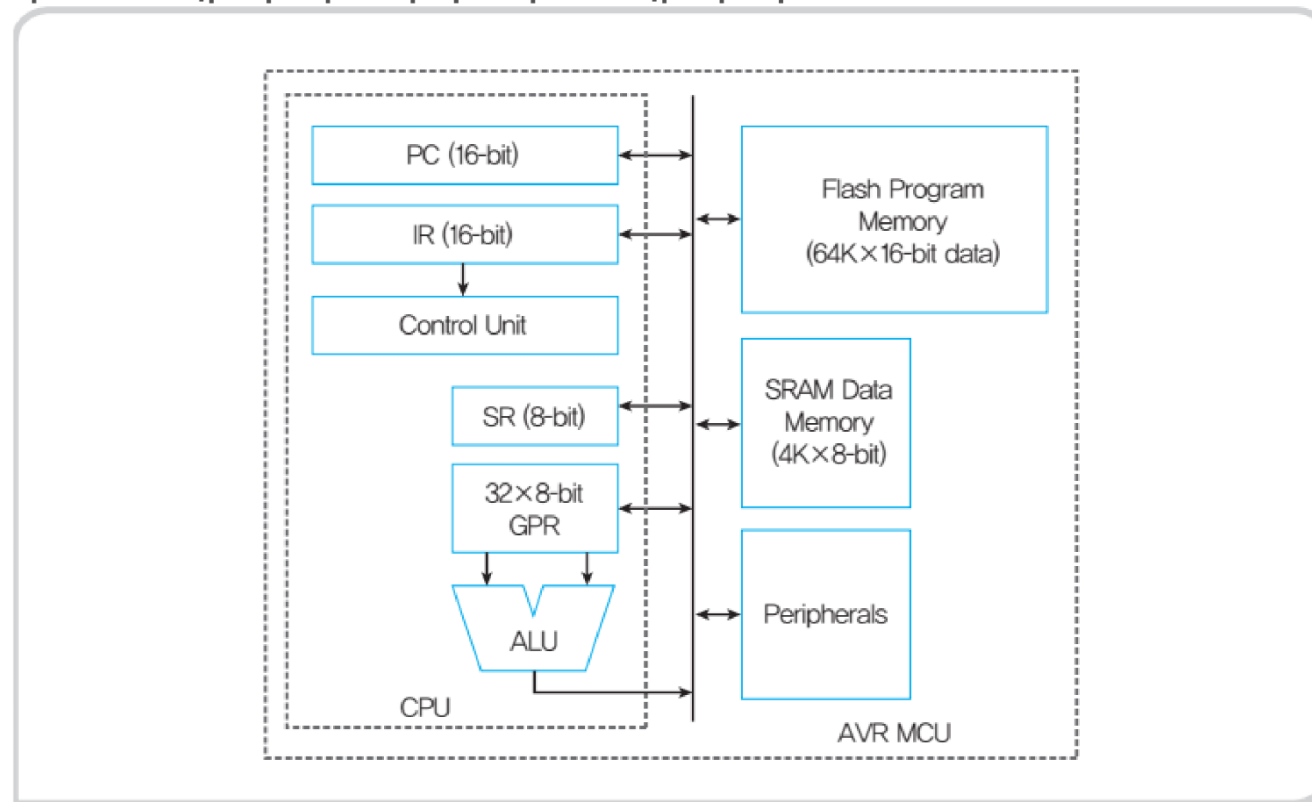


그림 3-8 AVR 계열 프로세서 구조

3.3 시스템 버스 구성요소를 연결하는 신호선의 모임

학습 목표

한개의 신호선은 한 비트 전달

- 컴퓨터의 세 가지 구성 요소를 연결하는 시스템 버스의 구성과 신호의 종류를 이해한다.

구성

- 3.3.1 시스템 버스 구성
- 3.3.2 제어 신호 종류
- 3.3.3 버스 계층

3.3.1 시스템 버스 구성

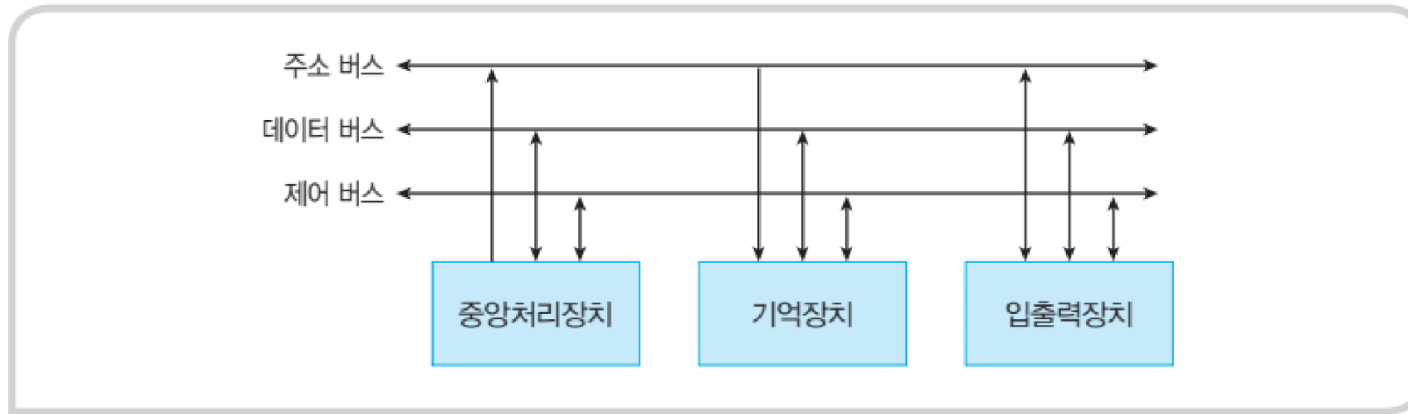


그림 3-9 시스템 버스

주소 버스

- 기억장치 또는 입출력장치의 주소를 지정한다.
- 기억장치 용량과 관련이 있다.

데이터 버스

- 데이터를 전달하는 경로이고, 일반적으로 8의 배수이다.
- 데이터 버스의 비트 수 = 레지스터의 크기 = 단어 크기(word size)

제어 버스

- 주소 버스와 데이터 버스를 제외한 나머지 신호.
- 각각 고유한 기능을 갖는다.

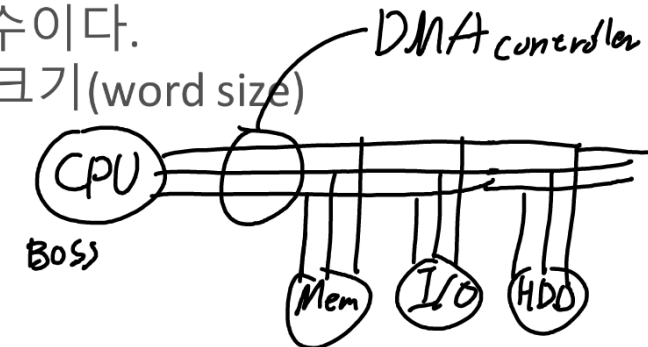
+) DMA Controller (중간버스 역할)

<데이터 입력>

데이터 입력 → CPU가 주소버스 구동 → 제어버스로
입출력장치 읽기 신호 보냄 → 해당 장치는 데이터 출력

<데이터 출력>

CPU가 주소버스 구동 → 데이터버스로 데이터 제공 →
제어버스로 입출력 쓰기 신호 보냄 → 해당 장치는 수신한
데이터 저장



3.3.2 제어 신호 종류

기억장치 제어

- 기억장치 읽기(memory read)
- 기억장치 쓰기(memory write)

입출력장치 제어

- 입출력 읽기(input 또는 I/O read)
- 입출력 쓰기(output 또는 I/O write)

✱ 인터럽트 요청(interrupt request)

✱ 인터럽트 확인(interrupt acknowledge)

- 버스 요청(bus request) DMA C 가 버스 사용권한 요청
- 버스 승인(bus grant) CPU가 DMAc 의 버스 사용 허가 통보

기타

- 시스템 클럭(system clock) 컴퓨터의 모든 동작 동기화
- 리셋(reset)
- 전력선(power lines)

3.3.3 버스 계층

버스 계층

- 버스 경쟁 완화 (장치가 많아지면 버스 경쟁 심화)
- 시스템 버스: 지역 버스, 고속 버스
- 입출력 버스: 확장 버스, 저속 버스, 장치 버스

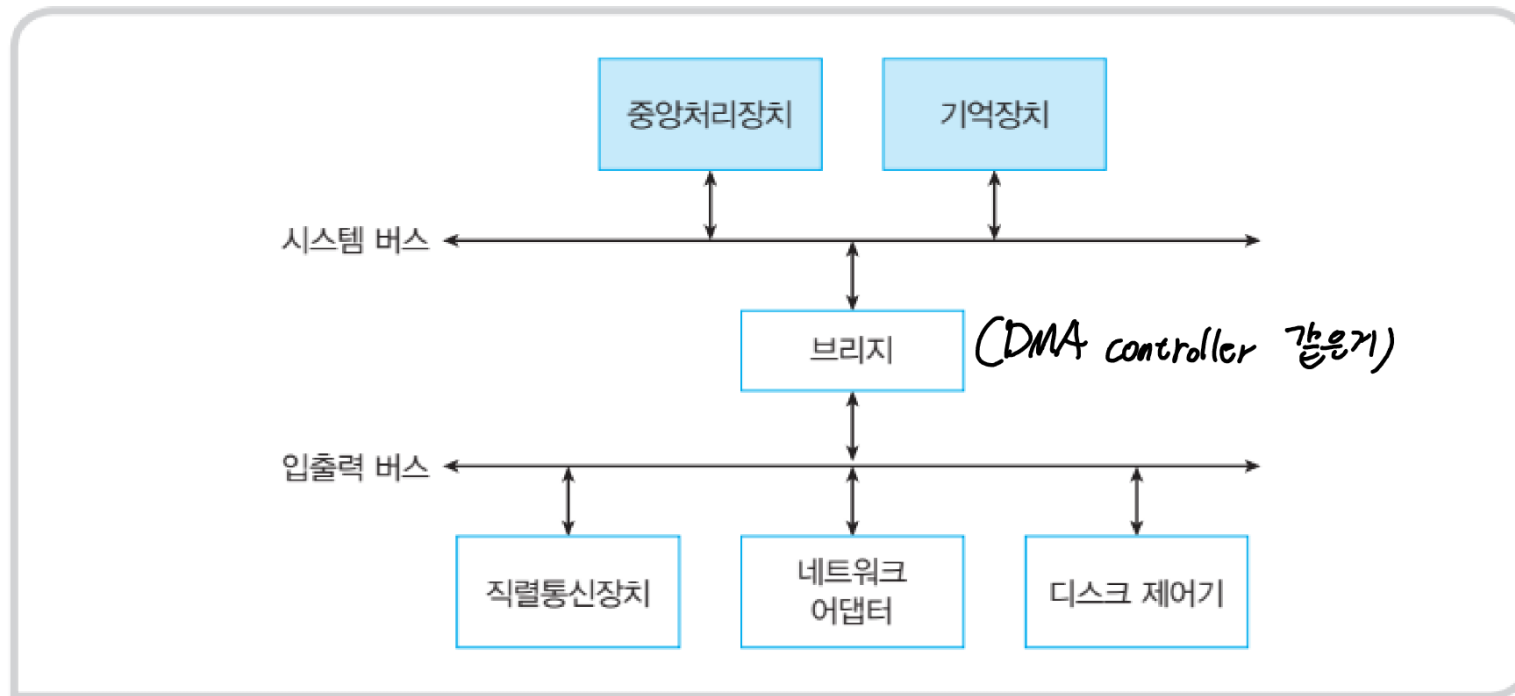


그림 3-10 버스 계층

3.4 명령어

명령어 집합(instruction set)

- 프로세서가 실행할 수 있는 명령어의 모임

학습 목표

- 명령어 구성 요소와 명령어를 이진수 코드로 표현하는 방법을 이해하고,
- 컴퓨터가 실행하는 명령어의 종류를 설명할 수 있다.

내용

- 3.4.1 명령어 구성 요소
- 3.4.2 명령어 종류
- 3.4.2 오퍼랜드

3.4.1 명령어 구성 요소

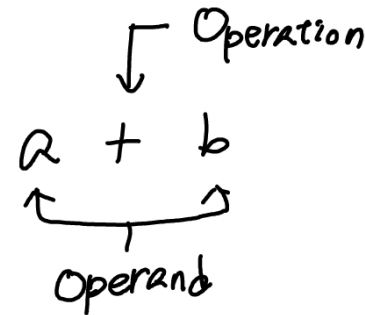
명령어

- 하드웨어와 소프트웨어가 만나는 부분
- 컴퓨터가 수행할 일을 2진수 코드로 표현한 것

구성 요소

- 동작 코드(opcode, operation code)
- 동작의 대상(operand): 최대 3개

피연산자



명령어 형식

- 명령어에 포함된 정보를 2진수에 배치한 형태
- 전체 길이, 필드 구성과 의미, 각 필드의 비트 수

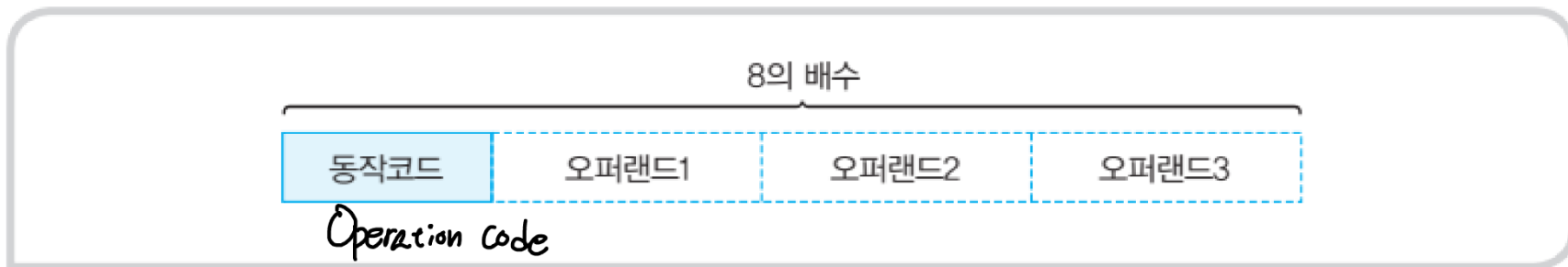


그림 3-11 명령어 형식

3.4.2 명령어 종류

동작 코드에 따른 명령어의 종류

- 데이터 전달 명령어(data transfer instructions)
 - 내부 레지스터, 주기억장치, 입출력장치 간 데이터 전달 (복사)
- 데이터 처리 명령어(data process instructions) *산술/논리 연산 수행*
 - 내부 레지스터 또는 주기억장치의 데이터를 연산기에서 조작
- 프로그램 제어 명령어(program control instructions)
 - 프로그램 실행 순서 변경 = 프로그램 카운터 변경
PC

```
int n = 1;           // 데이터 전달
int sum = 0;         // 데이터 전달
while (n <= 100) {   // 프로그램 제어
    sum = sum + n;    // 데이터 처리
    n = n + 1;        // 데이터 처리
}
```

데이터 전달 명령어

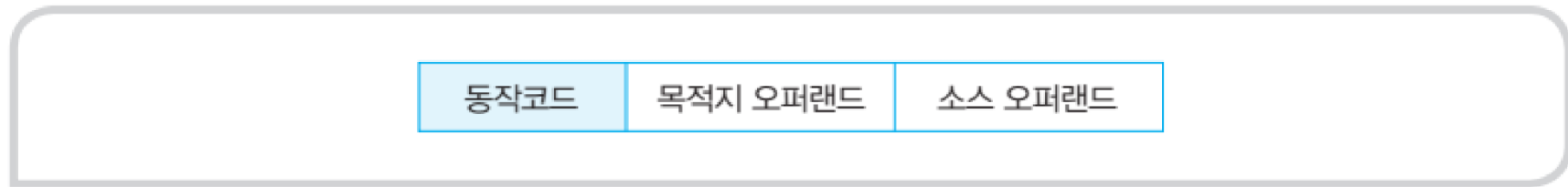


그림 3-12 데이터 전달 명령어 형식

- 적재(load): $\text{Register} \leftarrow \text{Mem}[\text{address}]$
- 저장(store): $\text{Mem}[\text{address}] \leftarrow \text{Register}$
- ✕ 이동(move): $\{\text{Register}, \text{Mem}[\text{address}]\} \leftarrow \{\text{Register}, \text{Mem}[\text{address}]\}$
- 입력(input): $\text{Register} \leftarrow \text{I/O Port}[\text{address}]$
- 출력(output): $\text{I/O Port}[\text{address}] \leftarrow \text{Register}$

데이터 처리 명령어

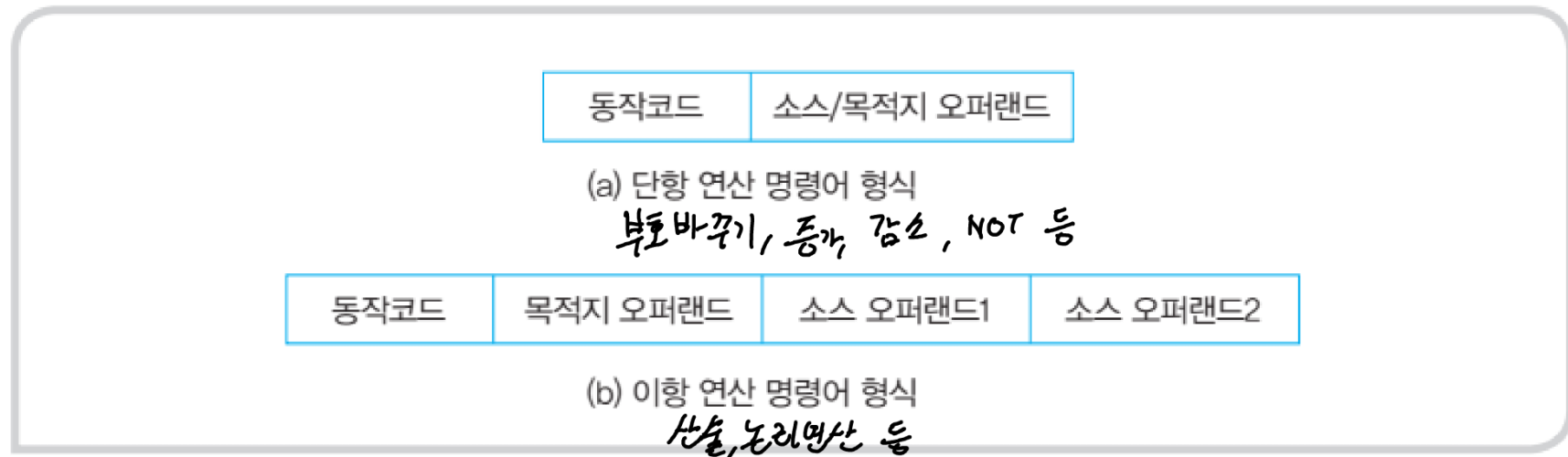


그림 3-13 데이터 처리 명령어 형식

- 목적지 = op 소스
- Negate (음수, 부호 변경)
- Not, Shift
- 증가, 감소
- 목적지 = 소스1 op 소스2
- 소스 오퍼랜드 한 개 생략 가능:
목적지 = 목적지 op 소스
- 사칙 연산: +, -, x, /
- 논리 연산: AND, OR, XOR

프로그램 제어 명령어

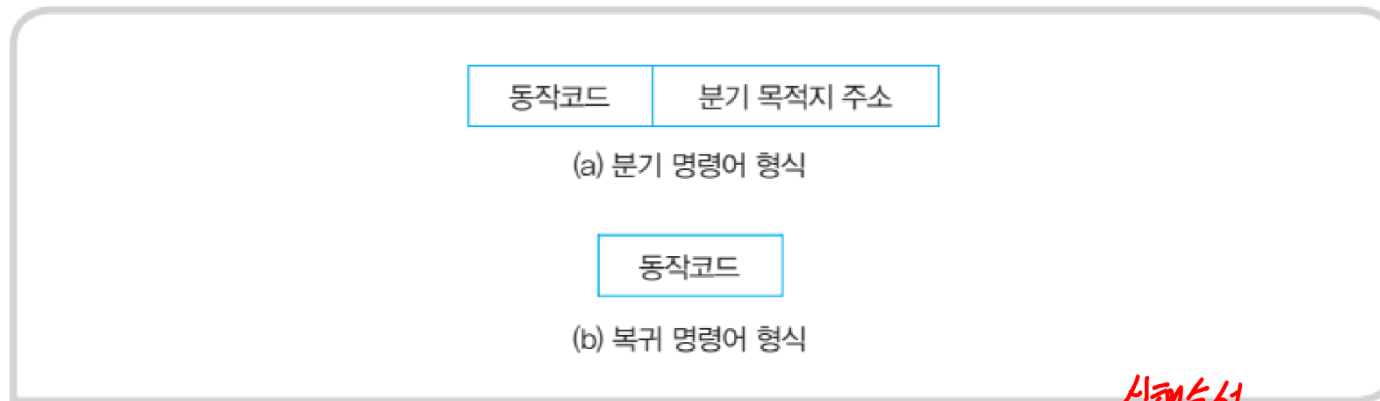
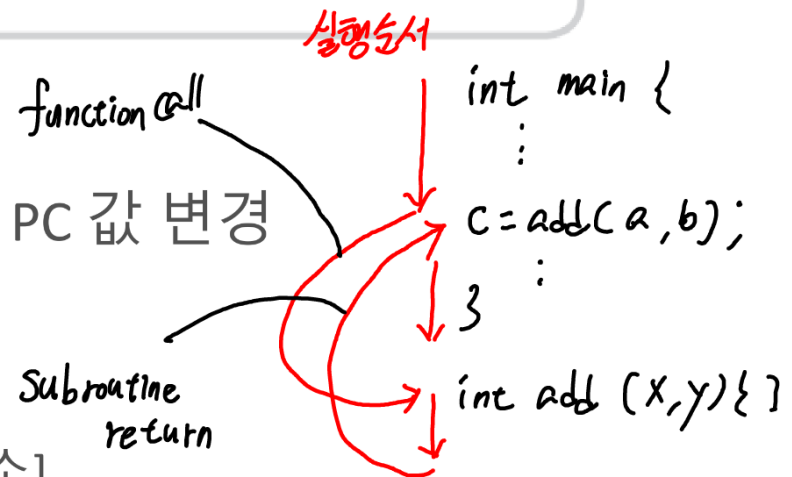


그림 3-14 프로그램 제어 명령어 형식

☆ 프로그램 실행 순서 변경: 궁극적으로 PC 값 변경



프로그램 제어 명령어 종류

- 무조건 분기:
- 조건 분기:
- 서브루틴 호출:
- 서브루틴 복귀:

PC ← [분기 주소]

if (조건) PC ← [분기 주소] ; if (), for () 등

PC 저장 / PC ← [서브루틴 시작 주소]

PC ← [저장된 주소]

Subroutine
= function

[예제]

[예제 3-3] 어느 그룹의 명령어인가?

- 1) $R1 \leftarrow \text{Mem}[400]$ (데이터 전달, 데이터 처리, 프로그램 제어)
- 2) $R2 \leftarrow R3 + \text{Mem}[401]$ (데이터 전달, 데이터 처리, 프로그램 제어)
- ✱ 3) $PC \leftarrow 1004$ (데이터 전달, 데이터 처리, 프로그램 제어)
- 4) $\text{Mem}[402] \leftarrow R2$ (데이터 전달, 데이터 처리, 프로그램 제어)
- ✱ 5) if (ZERO) $PC \leftarrow 1100$ (데이터 전달, 데이터 처리, 프로그램 제어)

3) and 5) : PC값이 바뀌므로 프로그램 제어

3.4.3 오퍼랜드

오퍼랜드 (operand, 피연산자)

- 명령어가 처리하려는 동작의 대상

유효 데이터 (effective data)

- 중앙처리장치가 실제로 처리하는 데이터
- 크기가 정해진 2진수 형태(예: 8 비트, 16 비트, 32 비트)

오퍼랜드 유형

- 즉치 데이터(immediate data): 명령어가 유효 데이터를 포함한다. 10, 100, 36 등
- 레지스터 이름: 레지스터의 값이 유효 데이터. PC, IR, R1, R2 등
- 기억장치 주소: 기억장치 해당 주소의 값이 유효 데이터.
- 입출력 포트: 입출력 포트의 값이 유효 데이터.

명령어 형식의 예

(1) $R3 \leftarrow R3 + 1$

동작코드

레지스터

INC	R3
-----	----

(2) $R3 \leftarrow R3 + 5$

동작코드

레지스터

즉치 데이터

ADD	R3	5
-----	----	---

→ 틀여쓰셈 ($R3 + = 5$ 같은 느낌)

(3) $\text{Mem}[1004] \leftarrow R1$

동작코드

기억장치 주소

레지스터

STORE	1004	R1
-------	------	----

(4) $R1 \leftarrow \text{Port}[1004]$

동작코드

레지스터

입출력 포트

INPUT	R1	1004
-------	----	------

(5) $R3 \leftarrow R1 + \text{Mem}[1004]$

동작코드

레지스터

레지스터

기억장치 주소

ADD	R3	R1	1004
-----	----	----	------

그림 3-15 명령어 형식의 예

동작코드	저장할 내용	오퍼랜드
------	--------	------

유효 데이터 해석

해석 방법

- 부호 없는 수 (unsigned number) *unsigned int*
- 정수 (signed number) *int*
- 실수 (floating-point number) *float*
- 문자 코드

명령어와 오퍼랜드의 관계

- 명령어에 따라 유효 데이터를 해석하는 방법이 다르다.
- 길이, 해석 방법이 다르면 명령어의 동작 코드도 달라야 한다.

3.5 명령어 실행 과정

학습 목표

- 간단한 프로그래머 모델의 동작을 이해한다.
- 중앙처리장치가 프로그램을 실행하는 과정을 설명할 수 있다.

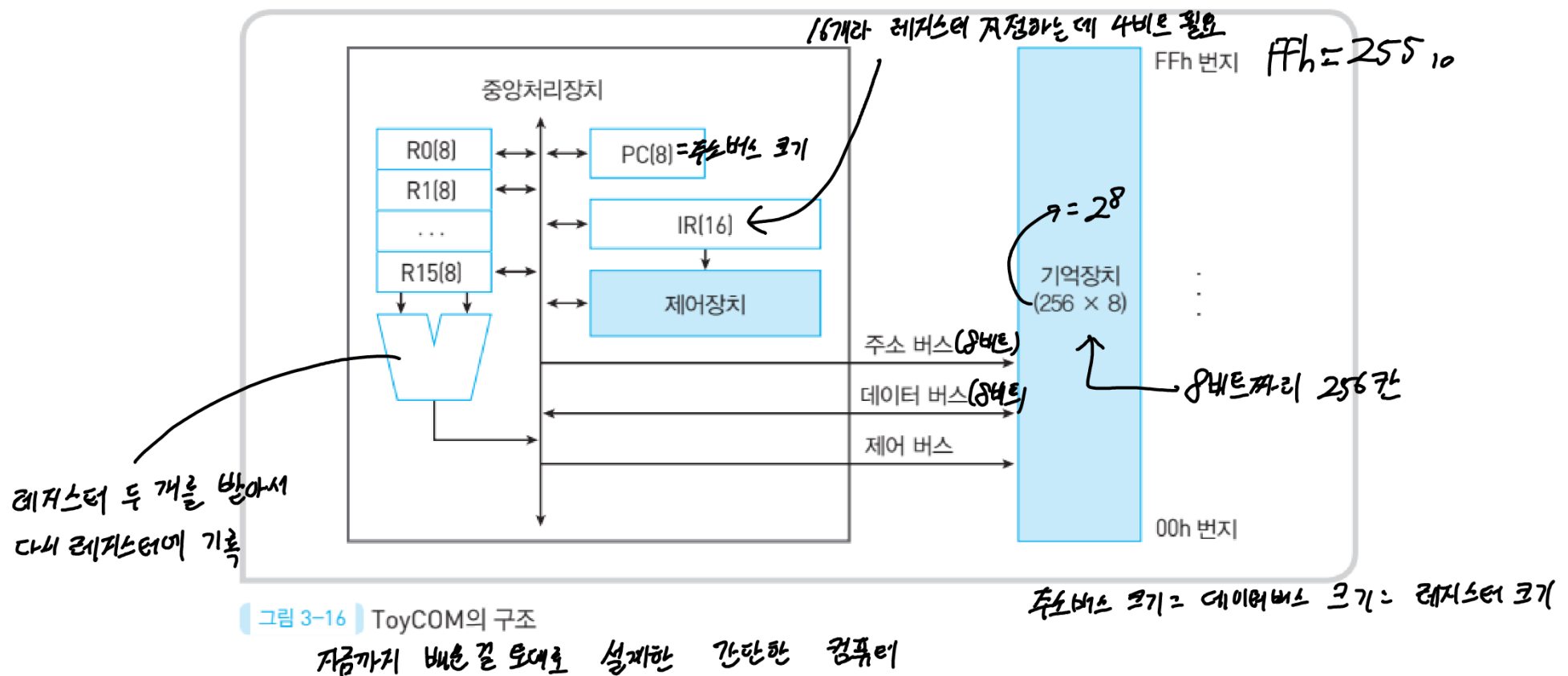
내용

- 3.5.1 프로그래머 모델
- 3.5.2 프로그램 실행 과정

3.5.1 프로그래머 모델

프로그래머 모델

- 어셈블러 프로그래머가 프로그램을 작성하기 위하여 알고 있어야 할 컴퓨터 구조



☆ ToyCOM 명령어 형식 ☆

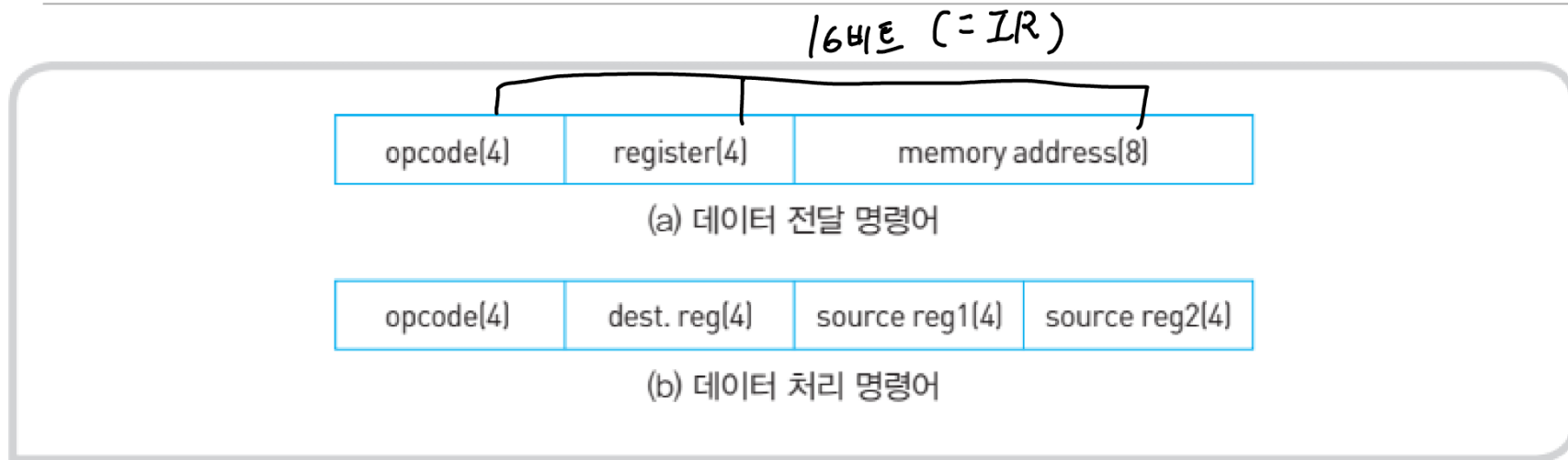


그림 3-17 ToyCOM 명령어 형식

명령어 종류

- 데이터 이동 명령어, 데이터 처리 명령어
- 프로그램 제어 명령어는 고려하지 않는다.

오퍼랜드

- 부호 없는 8 비트 데이터만 고려한다.

명령어 집합

표 3-1 ToyCOM 명령어 집합

동작	기호	동작코드	동작 설명
적재	load	0010 (2h)	기억장치 주소에서 레지스터로 데이터를 적재한다.
저장	store	0011 (3h)	레지스터의 값을 기억장치 주소로 저장한다.
더하기	add	1000 (8h)	소스 레지스터의 값을 더하여 목적지 레지스터에 저장한다.
빼기	sub	1001 (9h)	소스 레지스터의 값을 빼서 목적지 레지스터에 저장한다.
AND	and	1010 (ah)	소스 레지스터에 대하여 논리 AND 연산을 수행한다.
OR	or	1011 (bh)	소스 레지스터에 대하여 논리 OR 연산을 수행한다.

주소는 16진수
디폴트
h 바이트지마

예제 3-4) ToyCOM의 기계 명령어의 동작에 대한
어셈블리 언어와 레지스터 전송문을 구하라.

$\Rightarrow R2 \leftarrow \text{Mem}[10h]$

위 페이지 ToyCom 명령어 형식 참고

- Op code
- 1) 0010_0010_0001_0000 Read: 데이터 전달 $\Rightarrow$ Read R2 Mem[10h] $\Rightarrow$ load R2, Mem[10h]
 - 2) 0011_0011_0001_1000 Write $\Rightarrow$ Store R3, Mem[18h], Mem[18h] $\leftarrow$ R3
 - 3) 1000_0011_0011_0100 add R3, R3, R4 $R3 \leftarrow R3 + R4$
- assembly RTL

T₀ COM 명령어 : 16 bit
기억장치 Word 크기 : 8 비트 $\Rightarrow$ 한 개의 명령어는 기억장치 두 칸 점유

3.5.2 프로그램 실행 과정

명령어 사이클

- 인출 단계

$IR[7:0] \leftarrow Mem[PC], PC \leftarrow PC + 1$

$IR[15:8] \leftarrow Mem[PC], PC \leftarrow PC + 1$

- 실행 단계

- 명령어마다 다르다.

예제 프로그램

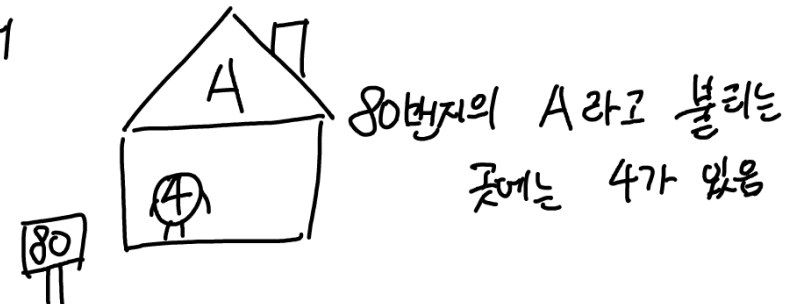
0010 - 0001 - 1000 - 0000
 ↓ ↓ ↓ ↓
 2 1 8 0 h

D = A + B - C 연산 전에 load

주소	어셈블리어	기계어 코드 (이진수)	16진수
	load R1, Mem[80h]	load R1 Mem[80h]	
00h	load R1, A	0010_0001_1000_0000	= 2180h
02h	load R2, B	0010_0010_1000_0001	2281h
04h	add R3, R1, R2	1000_0011_0001_0010	8312h
06h	load R1, C	0010_0001_1000_0010	2182h
08h	sub R3, R3, R1	1001_0011_0011_0001	9331h
0ah	store R3, D	0011_0011_1000_0011	3383h

...	참조데이터	실제데이터
→ 80h	A	4h
81h	B	6h
82h	C	3h
83h	D	0h

몇 번째 레지스터부터 변수가 저장되는 지는 다 다름
 To come 80h 부터



초기 상태

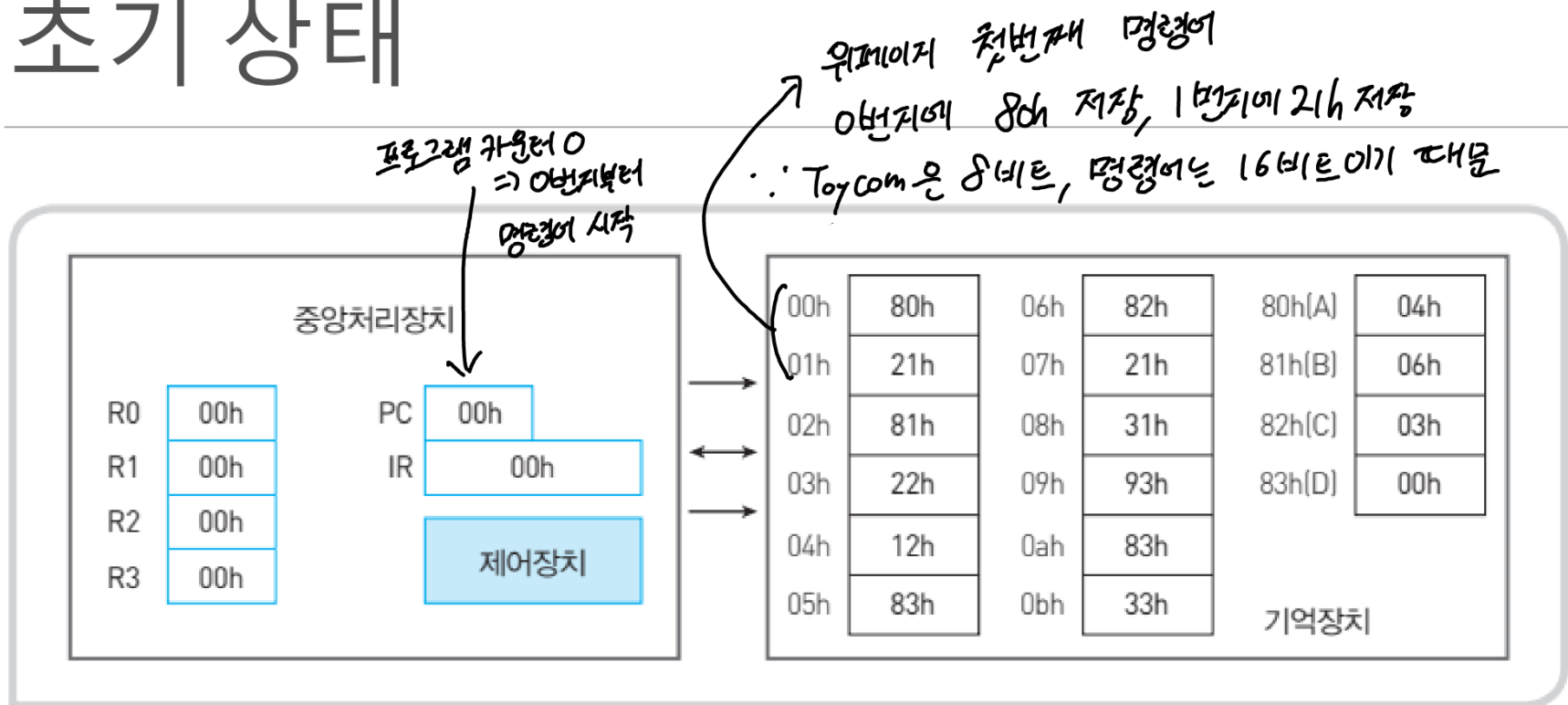


그림 3-18 중앙처리장치와 기억장치의 초기 상태

명령어 앞부분이 뒷주소예

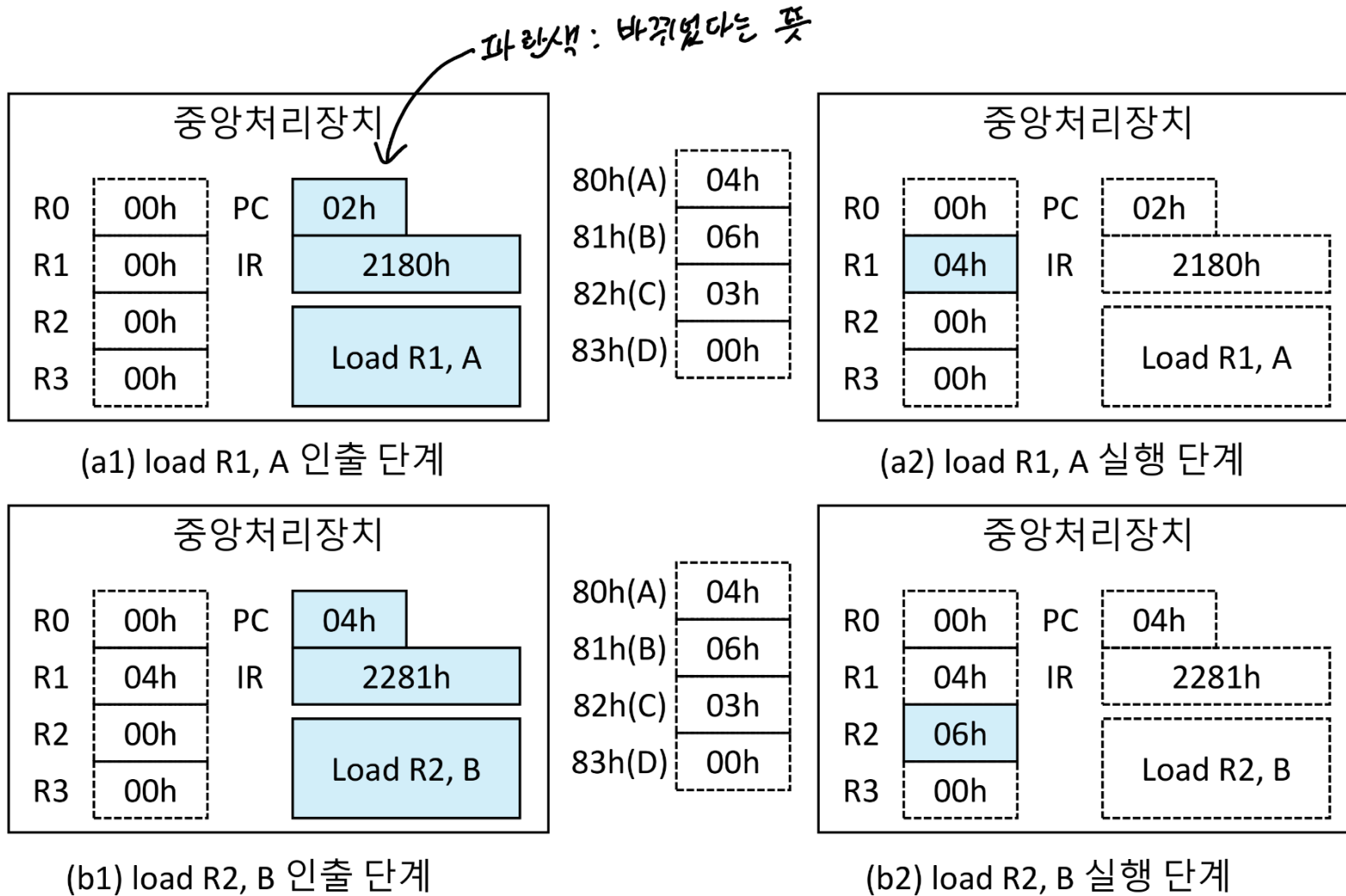
카운터가 하나씩 올라가기 때문

00h → 01h → 02h → ...

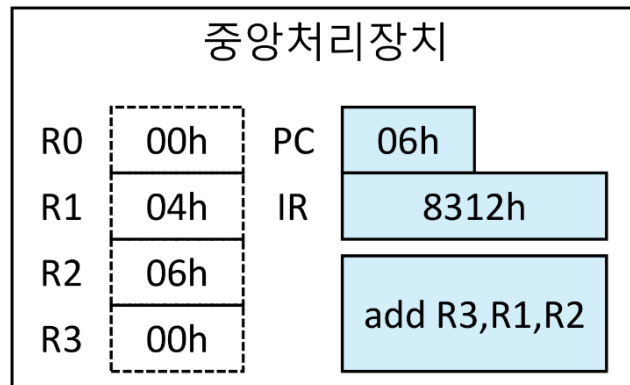
80h 21h 81h

↳ 21h80h

명령어 사이클

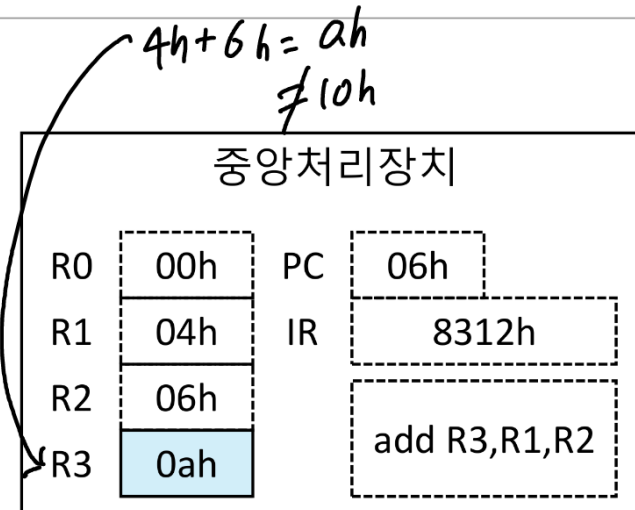


명령어 사이클

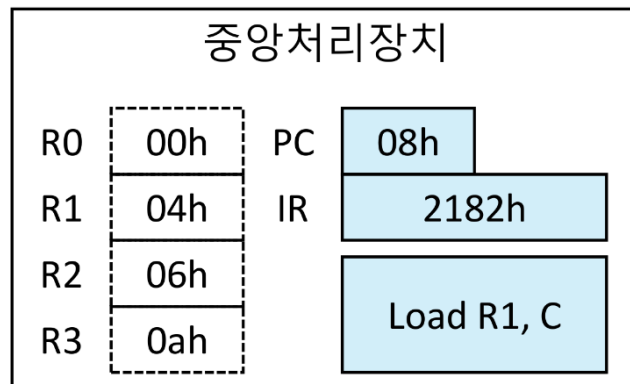


(c1) add R3,R1,R2 인출 단계

80h(A)	04h
81h(B)	06h
82h(C)	03h
83h(D)	00h

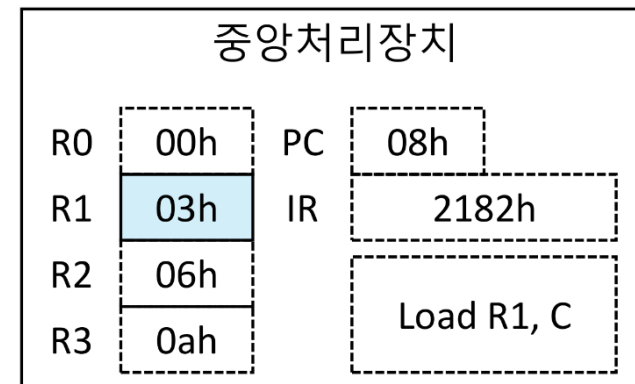


(c2) add R3,R1,R2 실행 단계



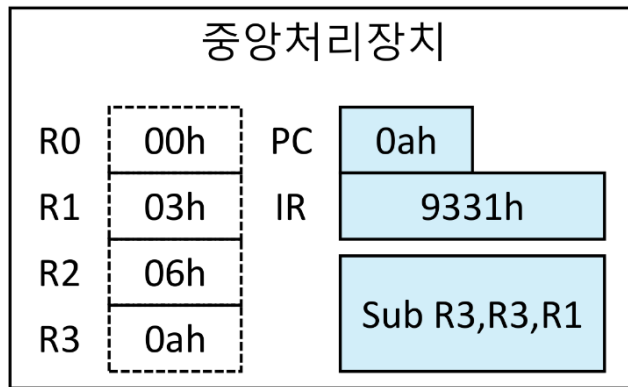
(d1) load R1, C 인출 단계

80h(A)	04h
81h(B)	06h
82h(C)	03h
83h(D)	00h



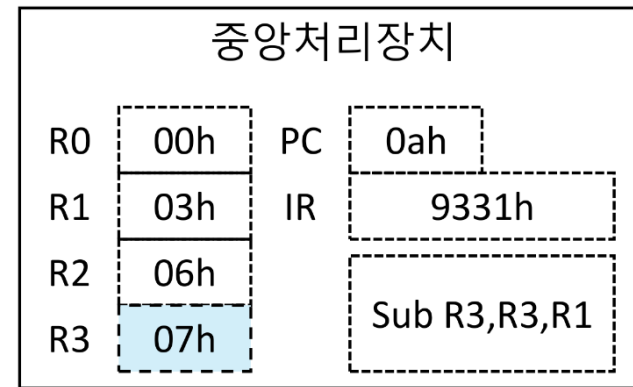
(d2) load R1, C 실행 단계

명령어 사이클

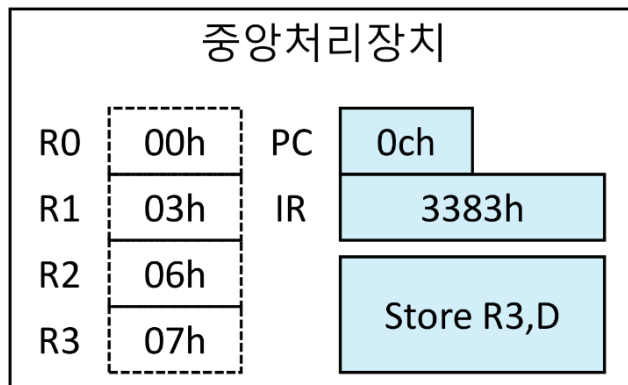


(e1) sub R3,R3,R1 인출 단계

80h(A)	04h
81h(B)	06h
82h(C)	03h
83h(D)	00h

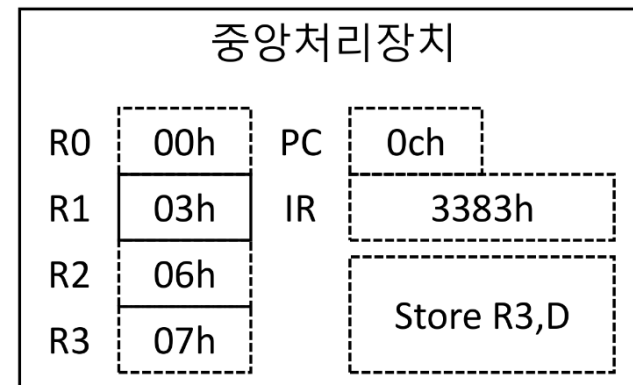


(e2) sub R3,R3,R1 실행 단계



(f1) store R3,D 인출 단계

80h(A)	04h
81h(B)	06h
82h(C)	03h
83h(D)	07h



(f2) load R1, C 실행 단계